Doubly Linked Lists in Python

A beginner-first textbook chapter

After this chapter, you should be able to implement a doubly linked list from scratch without fear, memorization, or pointer confusion.

Arrays → Classes → Singly Linked Lists → Doubly Linked Lists

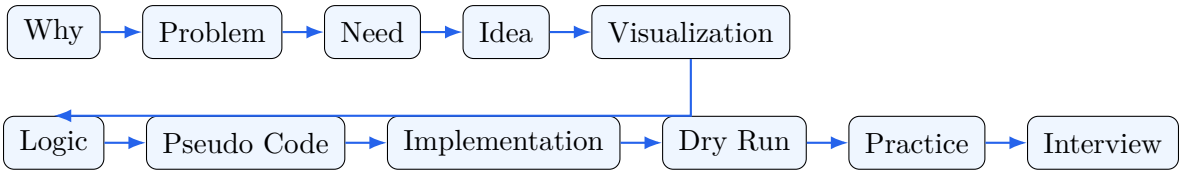
Contents

Chapter 1

Doubly Linked Lists in Python

How to Learn This Chapter

A doubly linked list is not difficult because the code is long. It feels difficult because many learners meet the code before they understand the pointer story. This chapter does the opposite. Every idea follows this path:



Tip

Do not try to memorize pointer updates. Instead, ask: *which relationship am I creating, which relationship am I removing, and what must not be lost before I change a reference?*

1.1 Revision: Singly Linked Lists

1.1.1 Why Linked Lists Exist

An array stores elements next to each other in memory. That is wonderful when we want fast access by index. If we know the starting address and the size of each element, the computer can jump directly to index 7.

But arrays become inconvenient when the collection grows, shrinks, or needs many insertions and deletions in the middle. If there is no free space beside the current array, the program may need to create a larger array somewhere else and copy items. If we insert at the beginning, many elements must shift.

A linked list takes a different approach: instead of requiring all elements to sit together, each value lives inside a small object called a **node**. Each node stores the address, or reference, of the next node.

Memory Picture

Array memory is like seats in one row:

`[10] [20] [30] [40]`

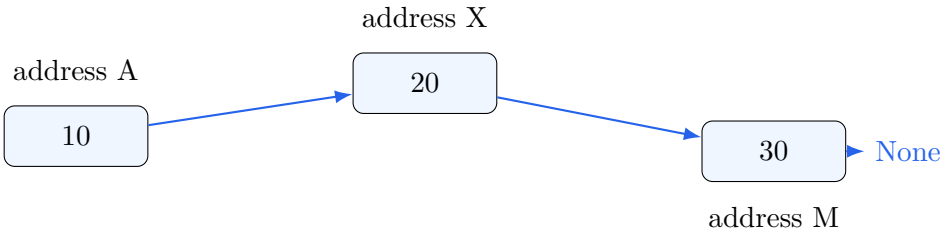
A singly linked list is like a treasure hunt:

`[10 | next] ---> [20 | next] ---> [30 | next] ---> None`

Each clue tells you where to go next.

1.1.2 Dynamic Memory

In a linked list, nodes do not need to be beside each other. Python may place them anywhere in memory. The list stays connected because each node keeps a reference to the next node.



The addresses are not important to us as numbers. The important point is: **a reference remembers where another object is**.

1.1.3 Node, Head, Tail, and Next

A singly linked list usually has these ideas:

- **Node**: stores data and a reference called `next`.
- **Head**: reference to the first node.
- **Tail**: optional reference to the last node.
- **Next pointer**: the direction from one node to the node after it.
- **Traversal**: moving from node to node using `next`.

Listing 1.1: A singly linked list node

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

1.1.4 Traversal in a Singly Linked List

Traversal means: start at `head`, repeatedly follow `next`, and stop at `None`.

Listing 1.2: Forward traversal pseudo code

```
current = head
WHILE current is not None:
    visit current.data
    current = current.next
```

Memory Picture

```
head
 |
 v
[10] ---> [20] ---> [30] ---> None
 ^
current starts here
```

After one move:

```
[10] ---> [20] ---> [30] ---> None
          ^
        current
```

1.1.5 Insertion and Deletion in a Singly Linked List

To insert after a node in a singly linked list, we connect the new node to the old next node, then connect the current node to the new node.

```
Before:
[10] ---> [30]

Insert 20 after 10:
[10] ---> [20] ---> [30]
```

To delete a node, the previous node must skip over it.

```
Before deleting 20:
[10] ---> [20] ---> [30]

After:
[10] ------------> [30]
```

Warning

Deletion is hard in a singly linked list because the node to be deleted does not know who points to it. If you stand on node 20, you can see node 30, but you cannot see node 10.

1.1.6 Conceptual Revision Questions

Pause and answer these before moving on.

1. Can we move backward from node 30 to node 20 using only `next`?
2. If we are standing on node 20, can we directly access node 10?
3. How many direction references does each singly linked list node store?
4. Why does deleting a node usually require knowing the previous node?
5. Can we insert before a given node easily if we only have that node?
6. Why is a tail reference useful for insertion at the end?
7. What happens if `head` is `None`?

1.2 The Problem With Singly Linked Lists

1.2.1 Real-Life Scenarios

Imagine a train. If every compartment only has a door to the next compartment, walking from the engine to the last coach is possible. But if you forget your bag in the previous coach, you cannot go back through a previous door. You must somehow start from the beginning again.

Now think about these systems:

- **Browser history**: Back and Forward buttons both matter.
- **Music playlist**: Previous song and next song both matter.
- **Photo gallery**: Swipe left and swipe right both matter.
- **Undo/redo**: Go to the previous state, then return to the next state.
- **Road navigation**: Sometimes the route must step backward.
- **Instagram stories**: You may tap left for the previous story and right for the next story.
- **Elevator buttons**: Move up or down depending on current position.

A singly linked list behaves like this:

```
Start ---> Page 1 ---> Page 2 ---> Page 3 ---> None
```

It supports *next*. But many real systems need *next and previous*.

Expert Thinking

The limitation is not that singly linked lists are bad. They are excellent when one-way movement is enough. The problem appears when the program repeatedly needs to move backward or modify relationships around a node.

1.3 Inventing the Doubly Linked List

Suppose we are standing at node 30 in a singly linked list:

```
[10] ---> [20] ---> [30] ---> None
                     ^
                  current
```

Can we go to 20? No. Why not? Because node 30 stores only one memory clue: the clue to the next node. It has forgotten where it came from.

What if every node stored one more reference: not only the next node, but also the previous node?

```
None <--- [10] <==> [20] <==> [30] ---> None
```

Now node 30 can say, "My previous node is 20." Node 20 can say, "My next node is 30 and my previous node is 10."

Definition

A **doubly linked list** is a linked list where each node stores three things: data, a reference to the next node, and a reference to the previous node.

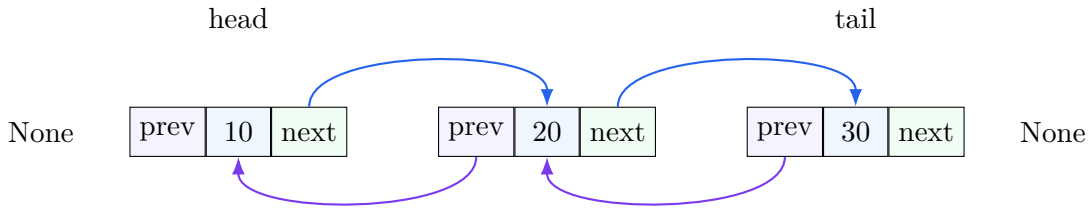
1.4 Visual Memory Representation

The standard picture is:

`None <- [10] <-> [20] <-> [30] -> None`

Every middle node has two arrows:

- `next`: points forward.
- `prev`: points backward.



1.4.1 Forward and Backward Movement

Forward movement follows `next`:

`head -> 10 -> 20 -> 30 -> None`

Backward movement follows `prev`:

`tail -> 30 -> 20 -> 10 -> None`

1.5 Node Structure

Before writing code, ask: what information must every node contain?

1. The actual value, such as 10.
2. A reference to the next node.
3. A reference to the previous node.

Listing 1.3: Doubly linked list node pseudo code

```
CREATE Node(value):
    data = value
    next = None
    prev = None
```

Listing 1.4: Doubly linked list node in Python

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5         self.prev = None
```

Line by line:

- `self.data = data`: stores the useful value.
- `self.next = None`: this node does not yet know who comes after it.

- `self.prev = None`: this node does not yet know who comes before it.

Memory Picture

Immediately after `Node(10)`:

```
prev    data    next
None <- [10] -> None
```

The node exists, but it is not connected to a list yet.

1.6 Building the Doubly Linked List Class

A list object manages the whole chain. It usually stores:

- `head`: first node.
- `tail`: last node.
- `size`: number of nodes.

Could we work without `tail`? Yes, but inserting at the end or traversing backward would require walking from the head to find the last node. Keeping `tail` gives direct access to the end.

Listing 1.5: List constructor pseudo code

```
CREATE DoublyLinkedList:
    head = None
    tail = None
    size = 0
```

Listing 1.6: DoublyLinkedList constructor

```
class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0
```

1.7 Traversal

1.7.1 Forward Traversal

Problem. Print or visit every node from first to last.

Idea. Start at `head`. Follow `next` until `None`.

Listing 1.7: Forward traversal pseudo code

```
current = head
WHILE current is not None:
    visit current.data
    current = current.next
```

Listing 1.8: Forward traversal code

```
def display_forward(self):
    values = []
    current = self.head
```

```
    while current is not None:
        values.append(str(current.data))
        current = current.next
    return " <-> ".join(values)
```

Complexity. Visits every node once, so time is $O(n)$ and extra space is $O(n)$ for the output list.

1.7.2 Backward Traversal

Problem. Visit every node from last to first.

Need. Singly linked lists cannot do this directly. Doubly linked lists can because every node has `prev`.

Listing 1.9: Backward traversal pseudo code

```
current = tail
WHILE current is not None:
    visit current.data
    current = current.prev
```

Listing 1.10: Backward traversal code

```
def display_backward(self):
    values = []
    current = self.tail
    while current is not None:
        values.append(str(current.data))
        current = current.prev
    return " <-> ".join(values)
```

1.8 Insertion Operations

Each insertion operation has one goal: place a new node into the chain without losing the existing chain.

1.8.1 Insert at Beginning

Problem. Add a new node before the current head.

Visualization.

```
Before: head -> [20] <-> [30] -> None
After:  head -> [10] <-> [20] <-> [30] -> None
```

Pointer changes.

1. New node's `next` points to old head.
2. Old head's `prev` points to new node.
3. `head` becomes new node.
4. If the list was empty, `tail` also becomes new node.

Warning

Order matters. If you replace `head` too early and forget the old head, you may lose the original list.

Listing 1.11: Insert first pseudo code

```
new_node = Node(value)
IF list is empty:
    head = new_node
    tail = new_node
ELSE:
    new_node.next = head
    head.prev = new_node
    head = new_node
size = size + 1
```

Listing 1.12: Insert first code

```
def insert_first(self, data):
    new_node = Node(data)
    if self.head is None:
        self.head = self.tail = new_node
    else:
        new_node.next = self.head
        self.head.prev = new_node
        self.head = new_node
    self.size += 1
```

Dry run. Insert 10 into `20 <-> 30`: create 10, connect 10.next to 20, connect 20.prev to 10, move head to 10.

Complexity. $O(1)$ time, $O(1)$ extra space.

Interview Note

A common interview check is whether you handle the empty-list case. If the first inserted node becomes `head` but not `tail`, backward traversal breaks.

1.8.2 Insert at End

Problem. Add a new node after the current tail.

Visualization.

```
Before: None <- [10] <-> [20] <- tail
After:  None <- [10] <-> [20] <-> [30] <- tail
```

Pointer changes.

1. New node's `prev` points to old tail.
2. Old tail's `next` points to new node.
3. `tail` becomes new node.
4. If empty, `head` also becomes new node.

Listing 1.13: Insert last pseudo code

```
new_node = Node(value)
IF list is empty:
    head = new_node
    tail = new_node
ELSE:
    new_node.prev = tail
```

```
    tail.next = new_node
    tail = new_node
size = size + 1
```

Listing 1.14: Insert last code

```
def insert_last(self, data):
    new_node = Node(data)
    if self.tail is None:
        self.head = self.tail = new_node
    else:
        new_node.prev = self.tail
        self.tail.next = new_node
        self.tail = new_node
    self.size += 1
```

Wrong order example.

```
self.tail = new_node
self.tail.next = new_node   # now tail is the new node, not the old tail
```

This creates a self-link or loses the connection from the old tail.

Complexity. $O(1)$ because `tail` gives direct access to the end.

1.8.3 Insert After a Node

Problem. Given a target value or node, insert a new node after it.

Visualization.

```
Before: [10] <-> [20] <-> [40]
Insert 30 after 20
After:  [10] <-> [20] <-> [30] <-> [40]
```

Pointer changes. Let `current` be node 20 and `after` be node 40.

1. Save `after = current.next`.
2. Set `new_node.prev = current`.
3. Set `new_node.next = after`.
4. Set `current.next = new_node`.
5. If `after` exists, set `after.prev = new_node`; otherwise update `tail`.

Listing 1.15: Insert after pseudo code

```
current = find target node
IF current does not exist:
    report failure
new_node = Node(value)
after = current.next
new_node.prev = current
new_node.next = after
current.next = new_node
IF after is not None:
    after.prev = new_node
ELSE:
    tail = new_node
size = size + 1
```

Listing 1.16: Insert after code

```
def insert_after(self, target, data):
    current = self.search(target)
    if current is None:
        return False

    new_node = Node(data)
    after = current.next
    new_node.prev = current
    new_node.next = after
    current.next = new_node

    if after is not None:
        after.prev = new_node
    else:
        self.tail = new_node

    self.size += 1
    return True
```

Mistake. If you do `current.next = new_node` before saving `after`, you lose access to the original next node unless another reference still remembers it.

Complexity. Searching is $O(n)$; actual insertion after the node is $O(1)$.

1.8.4 Insert Before a Node

Problem. Insert a new node before a target node.

Why doubly helps. Once we find the target node, `target.prev` directly tells us the node before it.

```
Before: [10] <-> [30] <-> [40]
Insert 20 before 30
After:  [10] <-> [20] <-> [30] <-> [40]
```

Listing 1.17: Insert before pseudo code

```
current = find target node
IF current does not exist:
    report failure
IF current is head:
    insert at beginning
ELSE:
    before = current.prev
    new_node = Node(value)
    new_node.next = current
    new_node.prev = before
    before.next = new_node
    current.prev = new_node
    size = size + 1
```

Listing 1.18: Insert before code

```
def insert_before(self, target, data):
    current = self.search(target)
    if current is None:
        return False
    if current is self.head:
```

```
        self.insert_first(data)
        return True

    before = current.prev
    new_node = Node(data)
    new_node.next = current
    new_node.prev = before
    before.next = new_node
    current.prev = new_node
    self.size += 1
    return True
```

Complexity. $O(n)$ to find the target, $O(1)$ to link the node.

1.8.5 Insert at Position

Problem. Insert at index 0, index `size`, or somewhere in the middle.

Listing 1.19: Insert at position pseudo code

```
IF index is invalid:
    raise error
IF index is 0:
    insert first
ELSE IF index is size:
    insert last
ELSE:
    current = node currently at index
    insert before current
```

Listing 1.20: Insert at position code

```
def insert_at(self, index, data):
    if index < 0 or index > self.size:
        raise IndexError("index out of range")
    if index == 0:
        self.insert_first(data)
        return
    if index == self.size:
        self.insert_last(data)
        return

    current = self._node_at(index)
    before = current.prev
    new_node = Node(data)
    new_node.prev = before
    new_node.next = current
    before.next = new_node
    current.prev = new_node
    self.size += 1
```

Dry run. Insert 25 at index 2 in `10,20,30`: node at index 2 is 30, before is 20, connect 20 and 30 through the new node 25.

1.9 Deletion Operations

Deletion means removing a node from the chain while reconnecting its neighbors.

1.9.1 Delete First

Problem. Remove the head node.

```
Before: head -> [10] <-> [20] <-> [30]
After:  head -> [20] <-> [30]
```

Listing 1.21: Delete first pseudo code

```
IF list is empty:
    raise error
removed = head
IF only one node:
    head = None
    tail = None
ELSE:
    head = head.next
    head.prev = None
size = size - 1
return removed.data
```

Listing 1.22: Delete first code

```
def delete_first(self):
    if self.head is None:
        raise IndexError("delete from empty list")
    removed = self.head
    if self.head is self.tail:
        self.head = self.tail = None
    else:
        self.head = self.head.next
        self.head.prev = None
    self.size -= 1
    return removed.data
```

Common bug. Forgetting `head.prev = None`. Then the new head still points backward to the removed node.

1.9.2 Delete Last

Listing 1.23: Delete last pseudo code

```
IF list is empty:
    raise error
removed = tail
IF only one node:
    head = None
    tail = None
ELSE:
    tail = tail.prev
    tail.next = None
size = size - 1
return removed.data
```

Listing 1.24: Delete last code

```
def delete_last(self):
    if self.tail is None:
```

```
        raise IndexError("delete from empty list")
    removed = self.tail
    if self.head is self.tail:
        self.head = self.tail = None
    else:
        self.tail = self.tail.prev
        self.tail.next = None
    self.size -= 1
    return removed.data
```

1.9.3 Delete by Value

Problem. Remove the first node whose data equals a target.

Listing 1.25: Delete value pseudo code

```
current = find target node
IF not found:
    return False
IF current is head:
    delete first
ELSE IF current is tail:
    delete last
ELSE:
    current.prev.next = current.next
    current.next.prev = current.prev
    size = size - 1
return True
```

Listing 1.26: Delete value code

```
def delete_value(self, target):
    current = self.search(target)
    if current is None:
        return False
    if current is self.head:
        self.delete_first()
    elif current is self.tail:
        self.delete_last()
    else:
        current.prev.next = current.next
        current.next.prev = current.prev
        self.size -= 1
    return True
```

1.9.4 Delete by Position

Listing 1.27: Delete at position pseudo code

```
IF index is invalid:
    raise error
IF index is 0:
    delete first
ELSE IF index is size - 1:
    delete last
ELSE:
    current = node at index
```

```
    current.prev.next = current.next
    current.next.prev = current.prev
    size = size - 1
    return current.data
```

Listing 1.28: Delete at position code

```
def delete_at(self, index):
    if index < 0 or index >= self.size:
        raise IndexError("index out of range")
    if index == 0:
        return self.delete_first()
    if index == self.size - 1:
        return self.delete_last()

    current = self._node_at(index)
    current.prev.next = current.next
    current.next.prev = current.prev
    self.size -= 1
    return current.data
```

1.9.5 Delete Before a Node

Listing 1.29: Delete before pseudo code

```
current = find target node
IF current does not exist OR current.prev is None:
    return False
node_to_delete = current.prev
IF node_to_delete is head:
    delete first
ELSE:
    node_to_delete.prev.next = current
    current.prev = node_to_delete.prev
    size = size - 1
return True
```

1.9.6 Delete After a Node

Listing 1.30: Delete after pseudo code

```
current = find target node
IF current does not exist OR current.next is None:
    return False
node_to_delete = current.next
IF node_to_delete is tail:
    delete last
ELSE:
    current.next = node_to_delete.next
    node_to_delete.next.prev = current
    size = size - 1
return True
```

1.9.7 Delete All Occurrences

Problem. Remove every node whose data equals a target. The key is to save `next_node` before deleting `current`; otherwise traversal may lose its path.

Listing 1.31: Delete all occurrences pseudo code

```
current = head
count = 0
WHILE current is not None:
    next_node = current.next
    IF current.data equals target:
        unlink current
        count = count + 1
    current = next_node
return count
```

1.10 Pointer Thinking

Experts do not think, "I hope these four lines are in the right order." They think in relationships.

1.10.1 The Four Most Useful Temporary Names

`current` The node we are standing on.

`previous` The node before `current`, often written as `current.prev`.

`next\_node` The node after `current`; save it before changing links.

`temp` A temporary safety reference used when something may be lost.

1.10.2 Reference Changes

A pointer update does not move a node. It changes which node a reference points to.

`current.next = new_node`

means: the `next` reference inside `current` now points to `new\_node`.

1.10.3 Garbage Collection and Lost Nodes

Python automatically frees objects when no active references can reach them. If you accidentally disconnect a node and keep no reference to it, Python may later collect it.

Warning

A lost node is not lost because it moved. It is lost because no variable or list pointer can reach it anymore.

1.10.4 Broken Chains

A correct middle connection must be true in both directions:

```
A.next is B
B.prev is A
```

If only one is true, forward and backward traversal disagree.

1.11 Common Beginner Mistakes

#	Mistake	Why it happens	Expert prevention
1	Forgetting to update `prev`	Learner thinks only forward	Check both directions
2	Forgetting to update `next`	Focuses on backward link	Draw before coding
3	Not updating `head`	New first node not registered	Handle head case first
4	Not updating `tail`	New last node not registered	Handle tail case first
5	Empty list not handled	Code assumes nodes exist	Ask "what if head is None?"
6	One-node delete breaks tail	Head cleared but tail remains	If head is tail, clear both
7	One-node delete breaks head	Tail cleared but head remains	Clear both endpoints
8	Losing original next node	Updated `current.next` too soon	Save `after` first
9	Losing original previous node	Updated `current.prev` too soon	Save `before` first
10	Incorrect size increment	Multiple branches increment twice	Increment once per success
11	Incorrect size decrement	Delete helper also decrements	Know who owns size update
12	Infinite traversal loop	A node points back through `next`	Verify final `tail.next=None`
13	Backward infinite loop	A node points forward through `prev`	Verify `head.prev=None`
14	Searching with `while current.next`	Last node skipped	Use `while current`
15	Deleting missing value crashes	Current becomes `None`	Check not found
16	Insert before head mishandled	`head.prev` is None	Delegate to insert first
17	Insert after tail mishandled	`tail.next` is None	Delegate or update tail
18	Confusing node and data	Compares node to value	Use `current.data`
19	Using `==` for node identity	Equal values confuse endpoints	Use `is` for same object
20	Negative index ignored	Python lists allow negatives, custom list may not	Define index policy
21	Index `size` rejected for insertion	Valid append position	Allow 0 through size
22	Index `size` allowed for deletion	No node exists there	Allow 0 through size-1
23	Forgetting return values	Caller cannot know result	Return clear success/data
24	Not testing two-node list	Edge behavior hidden	Test empty, one, two, many
25	Clearing only head	Tail still points to old chain	Clear head, tail, size
26	Reverse forgets head/tail swap	Direction changes but endpoints stale	Swap after reversing links
27	Iteration mutates list	Uses delete while yielding unsafely	Keep iteration read-only
28	Display crashes on empty	Joins uninitialized data	Return empty string

29	Insert creates node too late	Logic uses missing object	Create node before links
30	Insert creates node too early in failure path	Wasted object, confusing	Validate target first
31	Delete all skips nodes	Moves current after deletion incorrectly	Save `next\_node` first
32	Duplicate values surprise search	Search returns first match	Document behavior
33	Self-link accidentally created	Tail/head updated too early	Save old endpoint
34	Dangling backward reference	Removed node still referenced by new head prev	Null endpoint links
35	Middle delete forgets one side	Only bypasses forward	Reconnect both neighbors
36	Treating `None` as a node	Accessing `None.next`	Guard before access
37	Helper returns data not node	Insert needs references	Separate search and value access
38	No invariant checks	Bugs found too late	Use debugging assertions
39	Overcomplicated branches	Too many special cases	Reuse insert/delete helpers
40	Memorizing code	No mental model	Draw relationships first

1.12 Edge Case Masterclass

For every operation, ask which shape the list currently has.

Case	Diagram
Empty	`head=None, tail=None`
One node	`None <- [10] -> None`
Two nodes	`None <- [10] <-> [20] -> None`
Many nodes	`None <- [10] <-> [20] <-> [30] -> None`
Head node	Node whose `prev` is `None`
Tail node	Node whose `next` is `None`
Middle node	Has both `prev` and `next`
Duplicate nodes	More than one node has same data
Missing node	Search reaches `None`

Debugging Tip

Before finishing any operation, verify these invariants: `head.prev is None`, `tail.next is None`, forward count equals `size`, backward count equals `size`, and every neighboring pair agrees both ways.

1.13 Time Complexity

Operation	Array	Singly Linked List	Doubly Linked List
Access by index	$O(1)$	$O(n)$	$O(n)$
Search by value	$O(n)$	$O(n)$	$O(n)$

Insert beginning	$O(n)$ shift	$O(1)$	$O(1)$
Insert end	Often $O(1)$ amortized	$O(1)$ with tail	$O(1)$ with tail
Insert middle after node known	$O(n)$ shift	$O(1)$	$O(1)$
Insert before known node	$O(n)$ shift	Usually $O(n)$	$O(1)$
Delete first	$O(n)$ shift	$O(1)$	$O(1)$
Delete last	$O(1)$ often	$O(n)$ without prev	$O(1)$ with tail and prev
Delete known middle node	$O(n)$ shift	Needs previous	$O(1)$
Memory per item	Low	Data + next	Data + next + prev

1.14 Real-World Applications

- **Browser history**: current page has previous and next pages.
- **Undo/redo**: each editing state can point to the state before and after it.
- **Music player**: previous and next song navigation is natural.
- **Image viewer**: move left and right through photos.
- **Navigation systems**: route steps may be revisited in both directions.
- **Text editors**: cursor movement and editing structures often need neighboring positions.
- **Operating systems**: process and memory lists may need fast removal from known positions.
- **LRU cache**: combines a hash table with a doubly linked list to move recently used items quickly.
- **Deque**: double-ended queues need efficient operations at both ends.

1.15 Interview Thinking

Interviewers want to know whether you understand relationships, not whether you memorized lines.

Interview Note

When explaining, say: "I will handle empty and endpoint cases first. For a middle node, I reconnect the previous node to the next node and the next node back to the previous node. Then I update size."

Common questions:

1. Implement insert at head and tail.
2. Delete a node when only the node reference is given.
3. Reverse a doubly linked list.
4. Find pairs with a target sum in a sorted doubly linked list.
5. Explain why deletion is easier than in a singly linked list.
6. Compare memory cost with a singly linked list.

1.16 Complete Implementation

The final implementation below is intentionally clean and Pythonic. It includes helper methods but keeps pointer logic visible.

Listing 1.32: Complete doubly linked list implementation

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def __len__(self):
        return self.size

    def is_empty(self):
        return self.size == 0

    def __iter__(self):
        current = self.head
        while current is not None:
            yield current.data
            current = current.next

    def __str__(self):
        return " <-> ".join(str(value) for value in self)

    def _node_at(self, index):
        if index < 0 or index >= self.size:
            raise IndexError("index out of range")

        if index < self.size // 2:
            current = self.head
            for _ in range(index):
                current = current.next
        else:
            current = self.tail
            for _ in range(self.size - 1, index, -1):
                current = current.prev
        return current

    def display_forward(self):
        return " <-> ".join(str(value) for value in self)

    def display_backward(self):
        values = []
        current = self.tail
        while current is not None:
            values.append(str(current.data))
            current = current.prev
        return " <-> ".join(values)
```

```
53
54      def insert_first(self, data):
55          new_node = Node(data)
56          if self.head is None:
57              self.head = self.tail = new_node
58          else:
59              new_node.next = self.head
60              self.head.prev = new_node
61              self.head = new_node
62          self.size += 1
63
64      def insert_last(self, data):
65          new_node = Node(data)
66          if self.tail is None:
67              self.head = self.tail = new_node
68          else:
69              new_node.prev = self.tail
70              self.tail.next = new_node
71              self.tail = new_node
72          self.size += 1
73
74      def insert_after(self, target, data):
75          current = self.search(target)
76          if current is None:
77              return False
78
79          new_node = Node(data)
80          after = current.next
81          new_node.prev = current
82          new_node.next = after
83          current.next = new_node
84
85          if after is not None:
86              after.prev = new_node
87          else:
88              self.tail = new_node
89
90          self.size += 1
91          return True
92
93      def insert_before(self, target, data):
94          current = self.search(target)
95          if current is None:
96              return False
97          if current is self.head:
98              self.insert_first(data)
99              return True
100
101         before = current.prev
102         new_node = Node(data)
103         new_node.prev = before
104         new_node.next = current
105         before.next = new_node
106         current.prev = new_node
107         self.size += 1
108         return True
109
110     def insert_at(self, index, data):
```

```
        if index < 0 or index > self.size:
            raise IndexError("index out of range")
        if index == 0:
            self.insert_first(data)
            return
        if index == self.size:
            self.insert_last(data)
            return

        current = self._node_at(index)
        before = current.prev
        new_node = Node(data)
        new_node.prev = before
        new_node.next = current
        before.next = new_node
        current.prev = new_node
        self.size += 1

    def delete_first(self):
        if self.head is None:
            raise IndexError("delete from empty list")
        removed = self.head
        if self.head is self.tail:
            self.head = self.tail = None
        else:
            self.head = self.head.next
            self.head.prev = None
        self.size -= 1
        return removed.data

    def delete_last(self):
        if self.tail is None:
            raise IndexError("delete from empty list")
        removed = self.tail
        if self.head is self.tail:
            self.head = self.tail = None
        else:
            self.tail = self.tail.prev
            self.tail.next = None
        self.size -= 1
        return removed.data

    def delete_value(self, target):
        current = self.search(target)
        if current is None:
            return False
        if current is self.head:
            self.delete_first()
        elif current is self.tail:
            self.delete_last()
        else:
            current.prev.next = current.next
            current.next.prev = current.prev
            self.size -= 1
        return True

    def delete_at(self, index):
        if index < 0 or index >= self.size:
```

```
            raise IndexError("index out of range")
        if index == 0:
            return self.delete_first()
        if index == self.size - 1:
            return self.delete_last()

        current = self._node_at(index)
        current.prev.next = current.next
        current.next.prev = current.prev
        self.size -= 1
        return current.data

    def search(self, target):
        current = self.head
        while current is not None:
            if current.data == target:
                return current
            current = current.next
        return None

    def reverse(self):
        current = self.head
        while current is not None:
            current.prev, current.next = current.next, current.prev
            current = current.prev
        self.head, self.tail = self.tail, self.head

    def clear(self):
        self.head = None
        self.tail = None
        self.size = 0

    def to_list(self):
        return list(self)

    @classmethod
    def from_list(cls, values):
        linked_list = cls()
        for value in values:
            linked_list.insert_last(value)
        return linked_list
```

1.16.1 Method Explanations

- `\_node\_at`: chooses the shorter direction: from head for early indexes, from tail for late indexes.
- `search`: returns the first node with matching data, not the data itself.
- `reverse`: swaps `prev` and `next` inside every node, then swaps head and tail.
- `clear`: removes list access to all nodes; Python garbage collection handles unreachable nodes.
- `from\_list`: builds a linked list from normal Python values.

1.17 Visual Debugging

1.17.1 Insertion Before Middle Node

```
Memory before:
A:[10] <-> B:[30]

Create:
N:[20]

Pointer changes:
N.prev = A
N.next = B
A.next = N
B.prev = N

Memory after:
A:[10] <-> N:[20] <-> B:[30]
```

1.17.2 Deleting Middle Node

```
Before:
A:[10] <-> B:[20] <-> C:[30]

Delete B:
A.next = C
C.prev = A

After:
A:[10] <-> C:[30]
```

1.17.3 Reverse Animation

```
Original:
None <- [10] <-> [20] <-> [30] -> None

At node 10: swap prev and next
[10] now points backward to 20 and forward to None

At node 20: swap prev and next
At node 30: swap prev and next

Swap head and tail:
None <- [30] <-> [20] <-> [10] -> None
```

1.18 Practice

Practice

Easy exercises

1. Draw a doubly linked list containing 5, 10, 15.
2. Write pseudo code for forward traversal.
3. What are `head.prev` and `tail.next` always supposed to be?
4. Insert 1 at the beginning of an empty list by hand.
5. Explain why each node needs `prev`.

Practice

Medium exercises

1. Implement `delete_after(target)` in Python.
2. Implement `delete_before(target)` in Python.
3. Count how many times a value appears.
4. Find the middle node using two pointers.
5. Insert into a sorted doubly linked list while keeping it sorted.

Practice

Hard exercises

1. Remove duplicates from an unsorted doubly linked list.
2. Rotate the list right by `k` positions.
3. Check whether the list is a palindrome using head and tail pointers.
4. Implement a deque using a doubly linked list.
5. Implement a simple LRU cache using a dictionary and doubly linked list.

Practice

Debugging exercises

1. A backward display prints only the tail. Which pointer is probably missing?
2. Forward traversal never ends. Which endpoint invariant should you inspect?
3. After deleting the first node, backward traversal still shows it. What was forgotten?
4. Inserting after the tail works forward but not backward. What was forgotten?
5. Size says 4 but display shows 3 nodes. Where might size be updated incorrectly?

Practice

Pointer tracing For the list `10 <-> 20 <-> 30 <-> 40`, trace every pointer update for:

1. Insert 25 before 30.
2. Delete 20.
3. Insert 50 after 40.
4. Delete first.
5. Reverse the list.

1.19 Summary

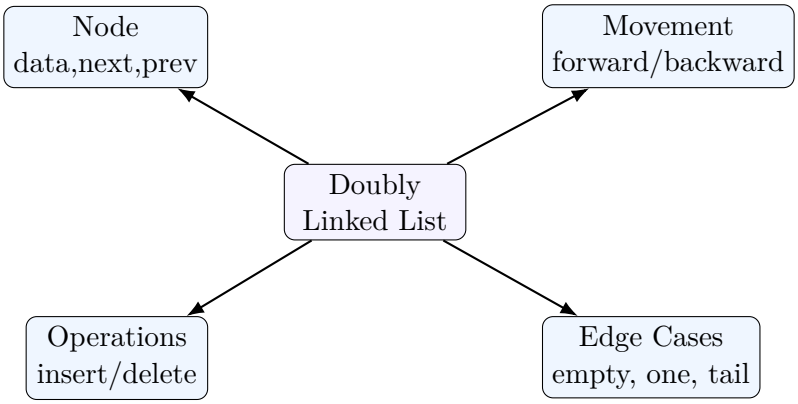
1.19.1 Cheat Sheet

- Node stores `data`, `next`, `prev`.
- `head` points to first node.
- `tail` points to last node.
- `head.prev` should be `None`.
- `tail.next` should be `None`.
- Forward traversal uses `next`.
- Backward traversal uses `prev`.
- Insertion creates relationships.
- Deletion removes relationships and reconnects neighbors.
- Search is still $O(n)$.

1.19.2 Pointer Update Summary

Operation	Core pointer idea
Insert first	New node points to old head; old head points back; move head
Insert last	New node points back to old tail; old tail points forward; move tail
Insert after	Save after; connect current, new node, and after
Insert before	Save before; connect before, new node, and current
Delete first	Move head forward; set new `head.prev=None`
Delete last	Move tail backward; set new `tail.next=None`
Delete middle	`current.prev.next=current.next`; `current.next.prev=current.prev`
Reverse	Swap `prev` and `next` in every node; swap head and tail

1.19.3 Mind Map



1.19.4 Flow Chart for Any Pointer Operation

1. Is the list empty?
2. Is the operation at the head?
3. Is the operation at the tail?
4. Otherwise, it is a middle-node operation.
5. Save any node that may become unreachable.
6. Update both forward and backward links.
7. Update `head`, `tail`, and `size` if needed.
8. Verify invariants by traversing forward and backward.

1.19.5 Final Memory Trick

Remember This

A doubly linked list is a hallway where every room has two doors: one to the next room and one to the previous room. Insertion means installing a new room and connecting both doors. Deletion means removing a room and reconnecting its neighbors' doors to each other.